LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL IV & V
ENCAPSULATION AND INHERITANCE

Disusun Oleh:

Michael Alexander Newton 105224017

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pemrograman Berorientasi Objek (PBO) merupakan pendekatan pemrograman yang mengorganisasikan perangkat lunak ke dalam unit-unit yang disebut objek, di mana setiap objek merupakan instansiasi dari suatu kelas (class) dan memiliki atribut serta perilaku tersendiri (Deitel & Deitel, 2018). Pendekatan ini menjadi landasan penting dalam rekayasa perangkat lunak masa kini, sebab memungkinkan penerapan konsep-konsep seperti enkapsulasi, pewarisan, dan kemampuan satu antarmuka merepresentasikan berbagai bentuk objek yang secara signifikan menyederhanakan pengelolaan sistem yang kompleks (Bloch, 2018).

Java adalah salah satu bahasa pemrograman berorientasi objek yang paling banyak digunakan di industri maupun akademik. Bahasa ini didesain dengan filosofi "write once, run anywhere" sehingga program yang dikompilasi menjadi bytecode dapat dijalankan di berbagai platform melalui Java Virtual Machine (JVM) tanpa modifikasi (Gosling et al., 2021). Hal tersebut menjadikan Java pilihan yang cocok dalam pembelajaran konsep dasar PBO, termasuk deklarasi class, pembuatan object, penerapan enkapsulasi, serta penggunaan constructor dan method overloading.

Praktikum Modul I ini berfokus pada implementasi konsep dasar Class dan Object dalam Java. Program yang dibuat adalah simulasi sistem manajemen kamar hotel dengan nama kelas KamarHotel. Program ini memodelkan entitas kamar hotel lengkap dengan atribut (field) dan perilaku (method) yang mencerminkan operasi nyata sebuah hotel. Melalui praktikum ini, diterapkan konsep enkapsulasi menggunakan modifier private dan public, pembuatan constructor dengan overloading untuk berbagai skenario inisialisasi objek, serta pembuatan method getter, setter, dan method fungsional seperti pesanKamar(), batalPesan(), dan hitungTotalBayar() (Savitch, 2018).

II. Variabel

Variabel pada class KamarHotel

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Menyimpan nomor identifikasi unik kamar hotel (contoh: "101", "102")
2	tipeKamar	String	Menyimpan tipe kamar hotel. Hanya menerima nilai "Reguler", "Premium", atau "Suite"
3	kapasitasMaksimal	int	Menyimpan jumlah maksimal tamu yang dapat menginap di kamar tersebut
4	hargaPerMalam	double	Menyimpan harga sewa kamar per malam dalam satuan Rupiah. Nilai minimum Rp 50.000
5	isTersedia	boolean	Menandai status ketersediaan kamar: true jika tersedia, false jika sudah dipesan

III. Constructor & Method

Constructor & Method pada kelas KamarHotel

No	Nama Constructor / Method	Return Type	Fungsi
1	KamarHotel(String, String, int)	Constructor	Constructor 3 parameter: menginisialisasi nomorKamar, tipeKamar, kapasitasMaksimal. hargaPerMalam diatur ke 0.0 dan isTersedia ke true
2	KamarHotel(String, String, int, double)	Constructor	Constructor 4 parameter: menginisialisasi semua atribut termasuk hargaPerMalam menggunakan setHargaPerMalam()
3	getNomorKamar()	String	Getter untuk mengembalikan nilai nomorKamar
4	getTipeKamar()	String	Getter untuk mengembalikan nilai tipeKamar
5	getKapasitasMaksimal()	int	Getter untuk mengembalikan nilai kapasitasMaksimal
6	getHargaPerMalam()	double	Getter untuk mengembalikan nilai hargaPerMalam
7	isTersedia()	boolean	Getter untuk mengembalikan status ketersediaan kamar
8	setTipeKamar(String)	void	Setter dengan validasi: hanya menerima "Reguler", "Premium", atau "Suite". Jika tidak valid, diatur ke "Reguler"
9	setHargaPerMalam(double)	void	Setter dengan validasi: harga minimum Rp 50.000. Jika kurang, otomatis diatur ke 50000.0
10	pesanKamar()	void	Method overload tanpa parameter: memesan kamar jika tersedia, mengubah isTersedia menjadi false
11	pesanKamar(int)	void	Method overload dengan parameter jumlahTamu: memesan kamar jika tersedia dan jumlah tamu tidak melebihi kapasitas
12	batalPesan()	void	Membatalkan pemesanan kamar jika kamar sedang dipesan, mengubah isTersedia kembali menjadi true
13	hitungTotalBayar(int)	double	Menghitung total biaya menginap: hargaPerMalam dikali jumlahMalam
14	hitungTotalBayar(int, String)	double	Overload hitungTotalBayar dengan kode voucher. Voucher "PROMO" memberikan diskon 20% jika jumlahMalam >= 3

No	Nama Constructor / Method	Return Type	Fungsi
15	tampilkanInfoKamar()	void	Menampilkan seluruh informasi kamar secara terformat ke konsol

IV. Dokumentasi dan Pembahasan Kode

4.1 Deklarasi Field dan Enkapsulasi

```
public class KamarHotel {

    private String nomorKamar, tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Seluruh field pada kelas KamarHotel dideklarasikan dengan access modifier private. Ini merupakan implementasi prinsip enkapsulasi dalam PBO, yaitu menyembunyikan detail internal objek dari luar kelas (information hiding). Dengan private, field-field tersebut tidak dapat diakses atau dimodifikasi langsung dari luar kelas, melainkan harus melalui method getter dan setter yang telah disediakan (Bloch, 2018).

Pemilihan tipe data menyesuaikan kebutuhan: String untuk data teks seperti nomorKamar dan tipeKamar, int untuk kapasitasMaksimal karena jumlah tamu selalu bilangan bulat, double untuk hargaPerMalam karena nilai uang bisa mengandung desimal, serta boolean untuk isTersedia karena status ketersediaan hanya memiliki dua kemungkinan nilai (Savitch, 2018).

4.2 Constructor Overloading

```
KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal){
        this.nomorKamar = nomorKamar;
        this.setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0.0;
        this.isTersedia = true;
    }

    KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal,
double hargaPerMalam){
        this.nomorKamar = nomorKamar;
        this.setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }
```

Kelas KamarHotel memiliki dua constructor yang berbeda parameter — ini disebut constructor overloading. Constructor pertama (3 parameter) digunakan saat harga belum ditentukan, sehingga hargaPerMalam diinisialisasi ke 0.0 sebagai nilai default. Constructor kedua (4 parameter) menerima harga secara langsung.

Kata kunci this digunakan untuk membedakan antara field milik objek dengan parameter yang memiliki nama sama. Perlu diperhatikan bahwa constructor memanggil setTipeKamar() dan setHargaPerMalam() daripada mengisi field secara langsung dengan tujuannya agar logika validasi dalam setter selalu dijalankan meskipun saat inisialisasi objek.

4.3 Setter dengan Validasi

```
public void setTipeKamar(String tipeKamar) {
        if(tipeKamar.equalsIgnoreCase("Reguler") ||
tipeKamar.equalsIgnoreCase("Premium") ||
tipeKamar.equalsIgnoreCase("Suite")){
            this.tipeKamar = tipeKamar;
        } else {
            System.out.println("Tipe kamar tidak valid. Tipe kamar diatur ke
Reguler.");
            this.tipeKamar = "Reguler";
        }
    }

    public void setHargaPerMalam(double hargaPerMalam) {
        if(hargaPerMalam < 50000.0){
            this.hargaPerMalam = 50000.0;
            System.out.println("Harga per malam tidak boleh kurang dari
50000.0. Harga diatur ke 50000.0.");
        } else {
            this.hargaPerMalam = hargaPerMalam;
        }
    }
```

Setter pada kelas ini tidak hanya berperan sebagai pintu masuk untuk mengubah nilai field, tetapi juga menjalankan validasi data. setTipeKamar() menggunakan equalsIgnoreCase() agar perbandingan string tidak sensitif terhadap huruf kapital, sehingga input "reguler" atau "REGULER" tetap diterima. Jika nilai tidak valid, tipeKamar secara otomatis diatur ke "Reguler" sebagai nilai fallback yang aman.

setHargaPerMalam() menerapkan batas minimum harga Rp 50.000. Jika harga yang diberikan lebih kecil (termasuk nilai negatif), sistem mengabaikan input tersebut dan menetapkan nilai minimum. Pola ini mencegah objek berada dalam keadaan data yang tidak logis (invalid state).

4.4 Method pesanKamar() dengan Overloading

```
public void pesanKamar() {
        if(isTersedia){
```

```
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
        } else {
            System.out.println("Maaf, kamar " + nomorKamar + " tidak
tersedia.");
        }
    }

    public void pesanKamar(int jumlahTamu){
        if(isTersedia){
            if(jumlahTamu <= kapasitasMaksimal){
                isTersedia = false;
                System.out.println("Kamar " + nomorKamar + " berhasil dipesan
untuk " + jumlahTamu + " tamu.");
            } else {
                System.out.println("Maaf, jumlah tamu melebihi kapasitas
maksimal kamar " + nomorKamar + ".");
            }
        } else {
            System.out.println("Maaf, kamar " + nomorKamar + " tidak
tersedia.");
        }
    }
```

Method pesanKamar() diimplementasikan dengan dua versi (method overloading). Versi tanpa parameter hanya mengecek ketersediaan kamar sebelum mengubah isTersedia menjadi false. Versi dengan parameter jumlahTamu menambahkan lapisan validasi kedua: selain cek ketersediaan, ia juga memastikan jumlah tamu tidak melebihi kapasitasMaksimal.

Struktur if bersarang (nested if) pada versi kedua mencerminkan urutan logika yang benar: mengecek ketersediaan terlebih dahulu baru kemudian mengecek kapasitas, karena tidak ada gunanya mengecek kapasitas jika kamar memang sudah tidak tersedia.

4.5 Method hitungTotalBayar() Overloading dan batalPesan()

```
public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equalsIgnoreCase("PROMO") && jumlahMalam >= 3) {
            System.out.println("Voucher PROMO diterima! Anda mendapatkan
diskon 20%.");
            return total * 0.8; // Diskon 20%
        } else {
```

```
            System.out.println("Peringatan: Penggunaan voucher ditolak
(syarat tidak terpenuhi).");
            return hitungTotalBayar(jumlahMalam);
        }
    }
public void batalPesan() {
        if(!isTersedia){
            isTersedia = true;
            System.out.println("Pemesanan kamar " + nomorKamar + " berhasil
dibatalkan.");
        } else {
            System.out.println("Kamar " + nomorKamar + " sudah tersedia,
tidak ada pemesanan yang perlu dibatalkan.");
        }
    }
```

hitungTotalBayar() juga diimplementasikan dengan overloading. Versi satu parameter menghitung total biaya secara langsung dengan perkalian. Versi dua parameter memanggil versi pertama untuk mendapatkan total dasar, lalu menerapkan logika voucher di atasnya — ini merupakan contoh baik dari code reuse dalam satu kelas.

Syarat voucher PROMO menggunakan operator logika AND (&&): kode harus "PROMO" DAN jumlahMalam harus minimal 3 malam. Jika salah satu syarat tidak terpenuhi, voucher ditolak dan harga normal dikembalikan. Method batalPesan() bekerja kebalikan dari pesanKamar(): hanya bisa dieksekusi saat kamar tidak tersedia (sedang dipesan), menggunakan operator negasi (!) pada kondisi isTersedia.

4.6 Kode Main dan Pengujian Objek

```
KamarHotel k1 = new KamarHotel("101", "Reguler", 2);
        System.out.println("--- Uji Kamar 1 ---");
        k1.setTipeKamar("Presidential");
        k1.setHargaPerMalam(-10000);


        KamarHotel k2 = new KamarHotel("102", "Premium", 2, 150000);
        System.out.println("--- Uji Kamar 2 ---");
        k2.pesanKamar(4);
        k2.pesanKamar(2);
        k2.pesanKamar();

        System.out.println("--- Uji Pembayaran ---");
        System.out.println("Tagihan Kamar 101: " + k1.hitungTotalBayar(2,
"PROMO"));
        System.out.println("Tagihan Kamar 102: " + k2.hitungTotalBayar(4,
"PROMO"));

        k1.tampilkanInfoKamar();
```

```
        k2.tampilkanInfoKamar();
```

Kode Main berfungsi sebagai driver program untuk menguji semua fungsionalitas kelas KamarHotel. Objek k1 dibuat menggunakan constructor 3 parameter, kemudian diuji dengan input tidak valid untuk setter (tipe "Presidential" dan harga negatif) — keduanya harus ditangani dengan benar oleh validasi di dalam setter.

Objek k2 menguji method pesanKamar() secara berurutan: pertama dengan jumlah tamu melebihi kapasitas (harus ditolak), lalu dengan jumlah tamu valid (berhasil), dan terakhir mencoba memesan lagi saat kamar sudah dipesan (harus ditolak). Pengujian hitungTotalBayar() memperlihatkan perbedaan perilaku voucher berdasarkan durasi menginap.

V. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

- Konsep enkapsulasi diterapkan dengan mendeklarasikan seluruh field sebagai private dan menyediakan akses melalui getter dan setter, sehingga integritas data objek selalu terjaga.
- Constructor overloading memungkinkan pembuatan objek dengan berbagai skenario inisialisasi tanpa mengubah nama kelas, meningkatkan fleksibilitas penggunaan kelas.
- Setter dengan validasi merupakan praktik terbaik dalam PBO karena memastikan objek tidak pernah berada dalam keadaan data yang tidak valid, terlepas dari nilai yang diberikan oleh pengguna.
- Method overloading pada pesanKamar() dan hitungTotalBayar() menunjukkan bahwa satu nama method dapat melayani berbagai kebutuhan berbeda berdasarkan parameter yang diberikan, membuat kode lebih intuitif dan mudah digunakan.
- Penggunaan kata kunci this penting dalam constructor dan setter untuk membedakan parameter lokal dengan field milik objek, terutama ketika keduanya memiliki nama yang sama.
- Pemisahan logika validasi ke dalam setter dan pemanggilan setter dari dalam constructor adalah pola desain yang baik karena menghindari duplikasi kode validasi.

VI. Daftar Pustaka

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Deitel, P., & Deitel, H. (2018). Java How to Program, Early Objects (11th ed.). Pearson Education.

Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D., & Bierman, G. (2021). The Java Language Specification: Java SE 17 Edition. Oracle America, Inc.

Oracle. (2023). Java SE 17 Documentation: java.io.PrintStream. Oracle America, Inc.

Savitch, W. (2018). Java: An Introduction to Problem Solving and Programming (8th ed.). Pearson Education.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: Java Basics I. Laboratorium Komputer, Program Studi Ilmu Komputer, Fakultas Sains dan Ilmu Komputer, Universitas Pertamina.